

CS3213: Foundations of Software Engineering

Software Testing Overview



Outline

- Why do we test?
- What is a test case?
- What are different testing levels?
- How does testing fit into agile and plan-driven development?
- How can we test the requirements?
- How can we create additional tests using knowledge about the implementation?

Why Testing?



Why Testing?



Finding bugs!

Why Testing?



Finding bugs!

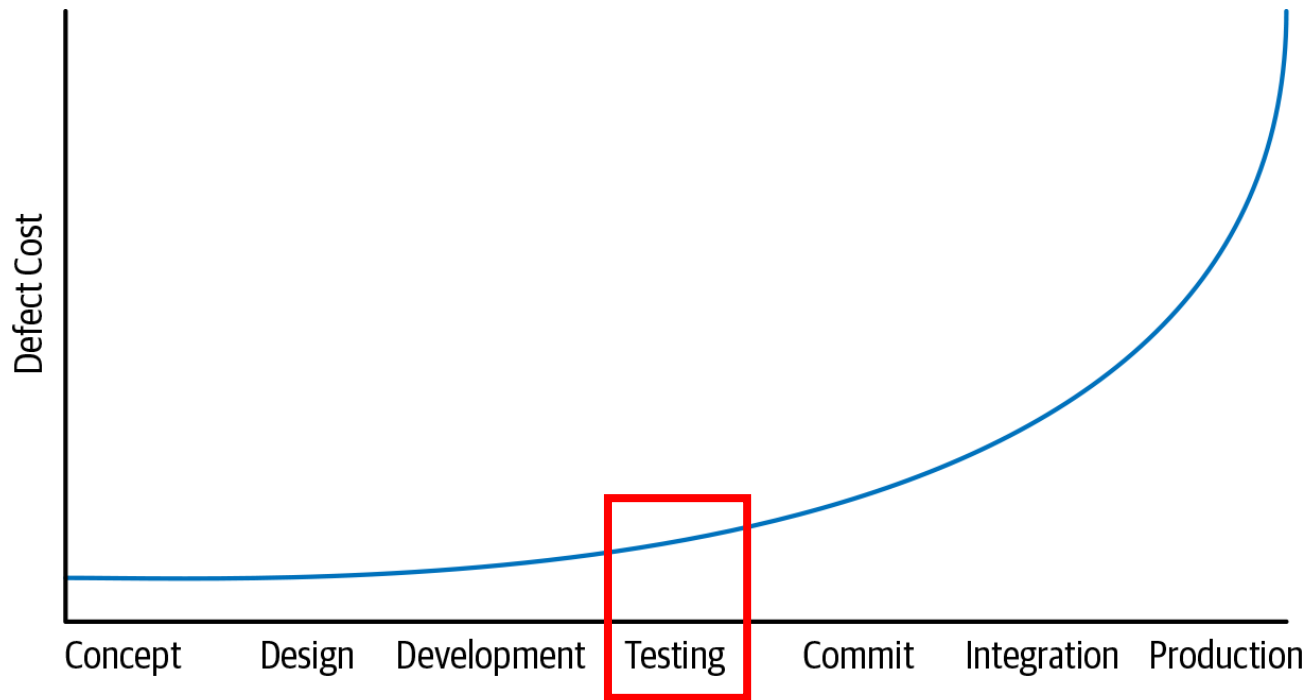
“Program testing can be used very effectively to show the presence of bugs but never to show their absence”
--- Edsger W. Dijkstra

Why Testing?

Validation testing!

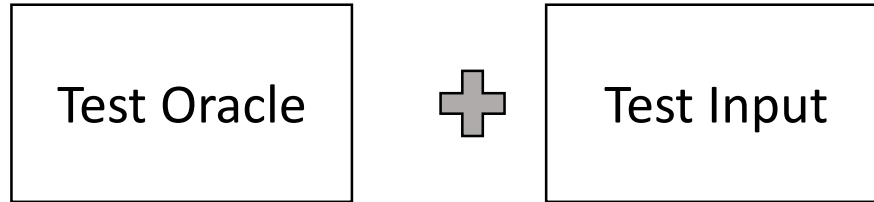


“Shifting Left”



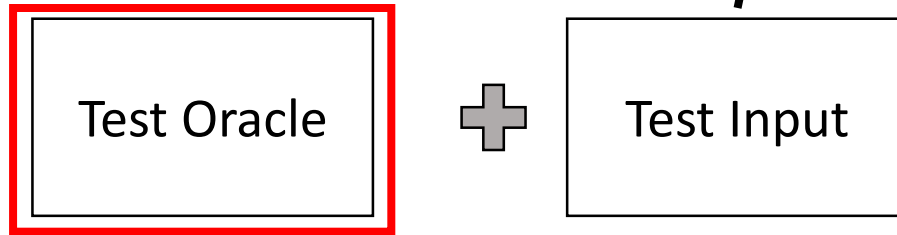
Test Case

```
@Test  
public void testAdd() {  
    assertEquals(5, add(2, 3));  
}
```

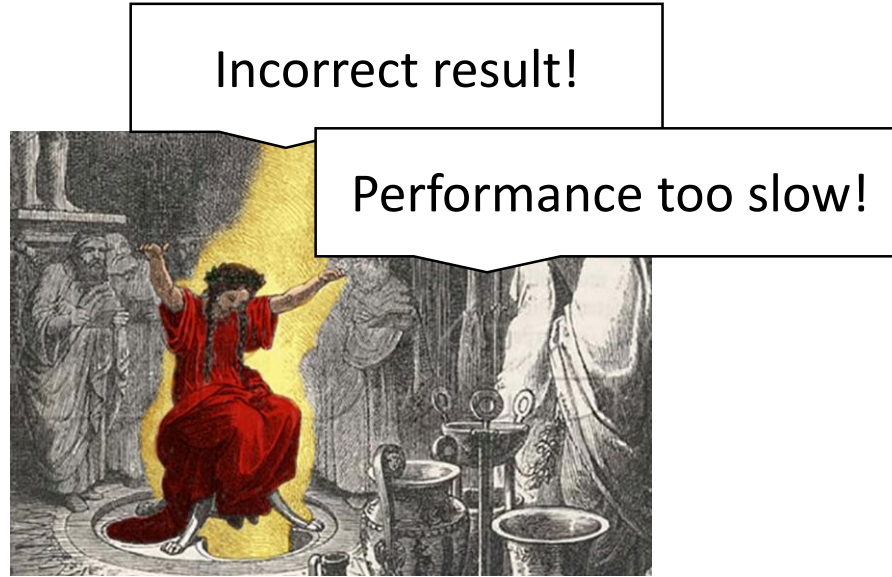


Test Case

```
@Test  
public void testAdd() {  
    assertEquals(5, add(2, 3));  
}
```



Test Oracle



“a test oracle (or just oracle) is a mechanism for determining whether a test has passed or failed”



Test Oracle

- Functional properties
 - Focus of this class!
- Non-functional properties
 - Performance
 - Usability
 - Accessibility
 - Security
 - ...

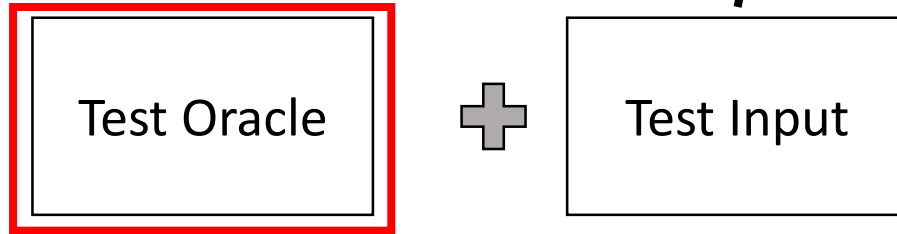
Test Oracle

- Functional properties
 - Focus of this class!
- Non-functional properties
 - Performance
 - Usability
 - Accessibility
 - Security
 - ...

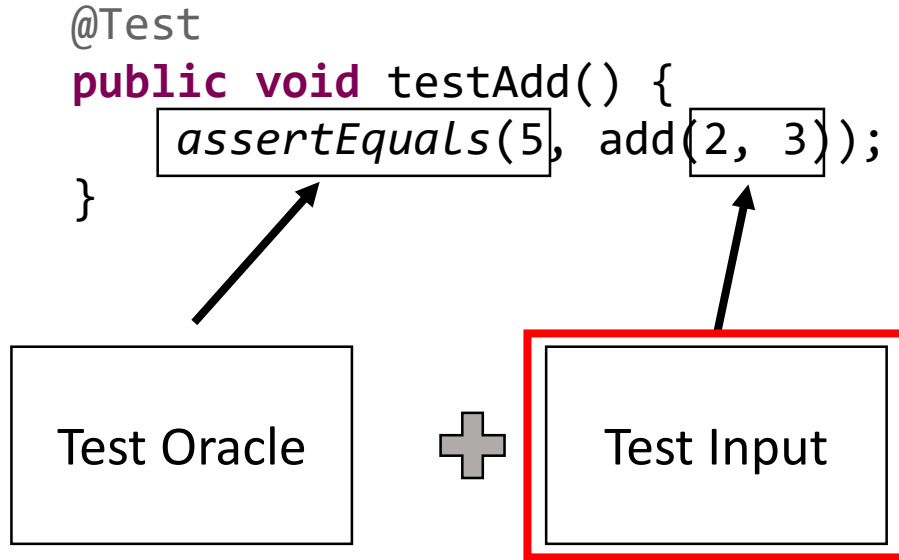
In this lecture, we will focus on **user-provided oracles** for a specific test input, known as **example-based testing**

Test Case

```
@Test  
public void testAdd() {  
    assertEquals(5, add(2, 3));  
}
```



Test Case





Test Inputs

- Arguments to a function
- System and environment state
- Sequence of actions
- Argument passed on the command line
- Button on a graphical user interface
- ...



Testing Levels

Unit Tests. Integration Tests. System Tests. Test sizes.



Testing Levels

- **Unit tests:** testing individual components
- **Integration tests:** testing multiple components in combination
- **System tests (or end-to-end tests):** evaluating the whole system

Unit Tests

```
public class PriceCalculator {  
    private final ShippingClient shippingClient;  
    private static final double GST_RATE = 0.09;  
  
    public PriceCalculator(ShippingClient shippingClient) {  
        this.shippingClient = shippingClient;  
    }  
  
    public double calculateTotal(double itemPrice, String postalCode) {  
        double shipping = shippingClient.getFee(postalCode);  
        return (itemPrice + shipping) * (1 + GST_RATE);  
    }  
}
```

Unit Tests: Mocking and Stubbing

```
@Test
void testCalculateTotal_WithMockedShipping() {
    // Mock the external dependency
    ShippingClient mockClient = mock(ShippingClient.class);
    when(mockClient.getFee("123456")).thenReturn(10.0);
    PriceCalculator calculator = new PriceCalculator(mockClient);

    // Execute the method we want to test
    double total = calculator.calculateTotal(100.0, "123456");

    // (100 + 10) * 1.09 = 119.9
    assertEquals(119.9, total, 0.001);
}
```



Unit Tests: Advantages

- Typically ***fast***, and allow testing of large portions of a system in limited time
- ***Easy to control***, as they typically check expected result values when passed a certain input value
- ***Easy to write***, as they require no additional set up (e.g., for databases or web services)



Unit Tests: Disadvantages

- Lack reality: does not represent the real execution of a system
- Some types of bugs might not be caught as they might be triggered only when integrating different components
- Might require mocks: simulate behavior of the “real” object



Integration Tests

- **Integration testing:** tests **multiple components of a system** together, focusing on the interactions between them instead of testing the system as a whole
- Typically focuses on two parts: “our” component, and an external component
- Example of external components: database system or web services

Integration Tests

```
@Test
void testIntegrationWithRealShippingService() {
    // real client (might connect to a test database/API)
    ShippingClient realClient = new ShippingClient("https://api.singpost.com/test");
    PriceCalculator calculator = new PriceCalculator(realClient);

    // We check if the wiring works and we get a valid response for a real SG postal code
    double total = calculator.calculateTotal(100.0, "129791");
    assertTrue(total > 100.0);
}
```

System Tests

- **System tests:** run the system **in its entirety**, instead of testing small parts in isolation

```
@Test
void testFullOrderFlow() {
    given()
        .body("{\"item\": \"[Same Day Delivery] Pahang Highland Musang King (3 boxes)(400g per box)\", \"postalCode\": \"129791\"}")
    .when()
        .post("/api/checkout")
    .then()
        .statusCode(200)
        .body("finalPrice", equalTo(128.00))
        .body("status", equalTo("READY_FOR_DELIVERY"));
}
```




System Tests: Advantages and Disadvantages

- Key advantage: tests are realistic
- Disadvantages
 - Often *slow compared to unit tests*, as it might involve multiple components
 - *Harder to write*, as some components (e.g., database systems) might require a complex setup
 - *Prone to flakiness*



Test Flakiness

- ***Flaky test***: a test that might non-deterministically pass or fail
- Lowers confidence in the tests, difficult to debug, might lower overall developer productivity



Test Flakiness: Reasons

- **Concurrency:** synchronization issues between threads
- **Async Wait:** asynchronous calls without properly waiting for their result to be come available
- **Test order dependency:** e.g., tests do not clean up
- Test case timeouts and **time-related issues**
- **Resource leaks**

Test Flakiness: Reasons

- **Concurrency:** synchronization issues between threads
- **Async Wait:** asynchronous calls without properly waiting for their result to be come available
- **Test order dependency:** e.g., tests do not clean up
- Test case timeouts and **time-related issues**
- **Resource leaks**

Test Flakiness: Concurrency

- Code below is executed by multiple threads
- Might throw a `ConcurrentModificationException`
- Potential solution: synchronized block

```
1 if (conf != newConf) {
2     for (Map.Entry<String, String> entry : conf) {
3         if ((entry.getKey().matches("hcat.*")) &&
4             (newConf.get(entry.getKey()) == null)) {
5             newConf.set(entry.getKey(), entry.getValue());
6         }
7     }
8     conf = newConf;
9 }
```



Test Flakiness: Reasons

- **Concurrency:** synchronization issues between threads
- **Async Wait:** asynchronous calls without properly waiting for their result to be come available
- **Test order dependency:** e.g., tests do not clean up
- Test case timeouts and **time-related issues**
- **Resource leaks**

Test Flakiness: Async Wait

- Whether the test case below succeeds depends on how quickly the server responds

```
1 @Test
2 public void testRsReportsWrongServerName() throws Exception {
3     MiniHBaseCluster cluster = TEST_UTIL.getHBaseCluster();
4     MiniHBaseClusterRegionServer firstServer =
5         (MiniHBaseClusterRegionServer)cluster.getRegionServer(0);
6     HServerInfo hsi = firstServer.getServerInfo();
7     firstServer.setHServerInfo(...);
8
9     // Sleep while the region server pings back
10    Thread.sleep(2000);
11    assertTrue(firstServer.isOnline());
12    assertEquals(2,cluster.getLiveRegionServerThreads().size());
13    ... // similarly for secondServer
14 }
```

Testing Granularities





Pesticide Paradox

- ***Pesticide Paradox***: “Every method you use to prevent or find bugs leaves a residue of subtler bugs against which those methods are ineffectual.”
 - Similar to insects/pests building up an immunity
- Different testing kinds or approaches have different strengths and weaknesses, so ideally, multiple/many different kinds are combined

Pesticide Paradox in Action (2025)

The verdict

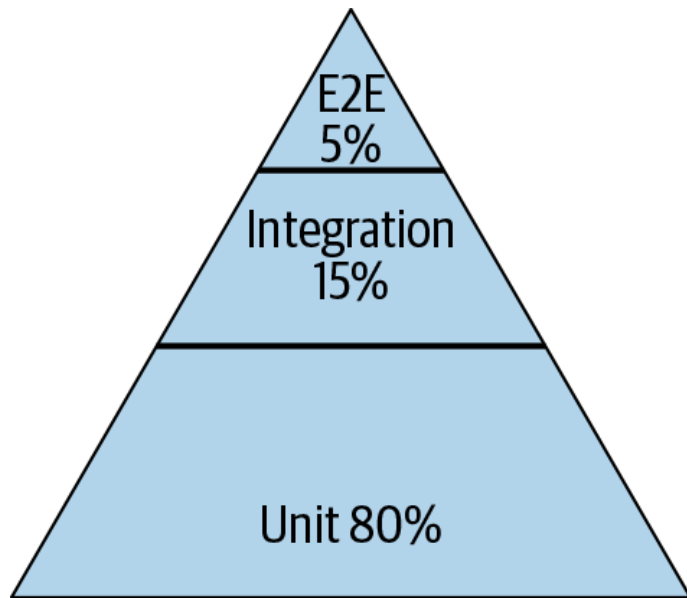
- There won't be a fuzzer that finds all the issues.
- Fuzzers follow the law of diminishing return.
- I can add more features to BuzzHouse but:
 - The codebase becomes more complex, then more subjective.
 - Finding issues in fuzzers is not trivial, because it's not clear what to look for.
 - More combinations, means less probability to find corner bugs.

Use a combination of fuzzers

- AFL and libFuzzer for code coverage-guided fuzzing.
- SQLsmith (<https://github.com/anse1/sqlsmith>) for large query generation.
- SQLancer (<https://github.com/sqlancer/sqlancer>) for query correctness.
- Pstress (<https://github.com/Percona-QA/pstress>) for heavy load.
- Sysbench (<https://github.com/akopytov/sysbench>) for long workloads (>1h).
- AST fuzzer to mutate queries from tests, or inputs from other fuzzers.
- BuzzHouse for randomly generated catalog-backed queries.

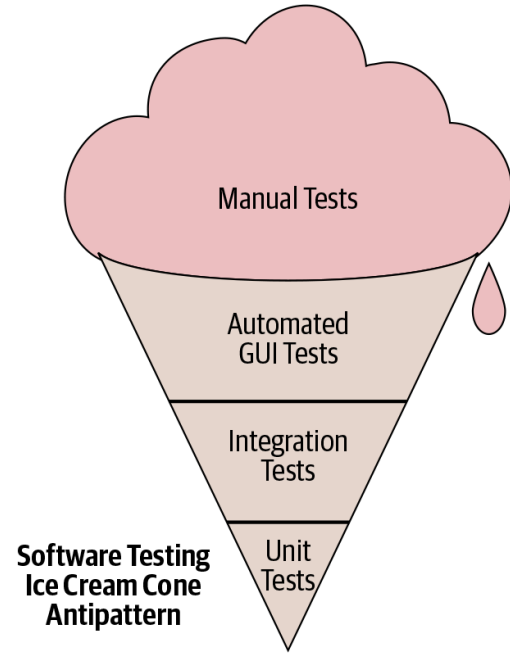
Google's Test Pyramid

- Write tests with different granularity
- Fewer high-level tests
- Many small and fast unit tests



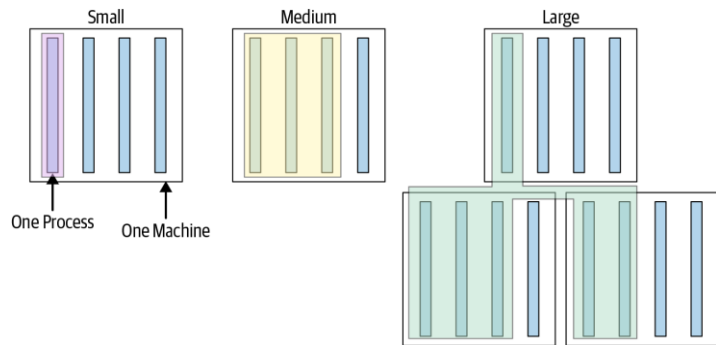
Pyramid Antipatterns

- Difficult to maintain
- Takes long to run
- Requires more debugging effort



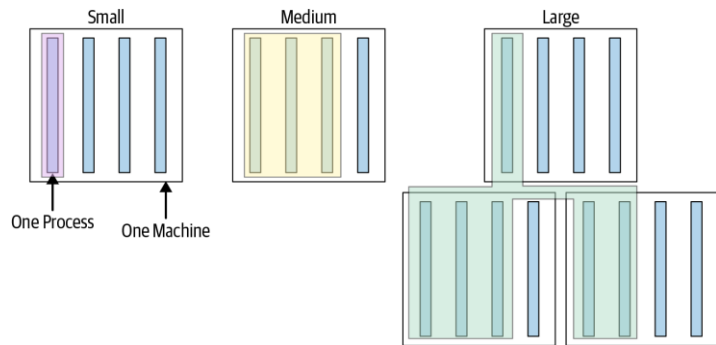
Related: Test Sizes at Google

- Google classifies every test into a size, encouraging engineers to always write the smallest test possible
- Test size limits what the test is allowed to do
- Goal: efficient execution and avoiding non-determinism (i.e., flaky tests)



Related: Test Sizes at Google

- **Small:** single process (or even thread)
 - Not allowed to sleep, no I/O operations
 - Fast and deterministic
- **Medium:** multiple processes
 - Blocking calls, including network calls, to localhost
- **Large:** remove the localhost restriction



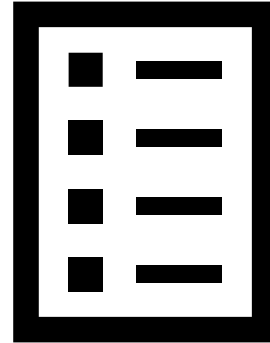


Testing and Processes

Requirements Specification



Agile

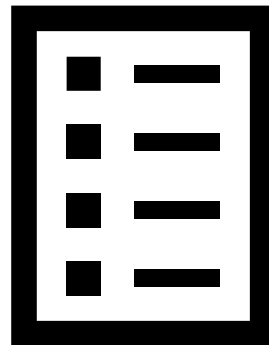


Plan-driven

Requirements Specification

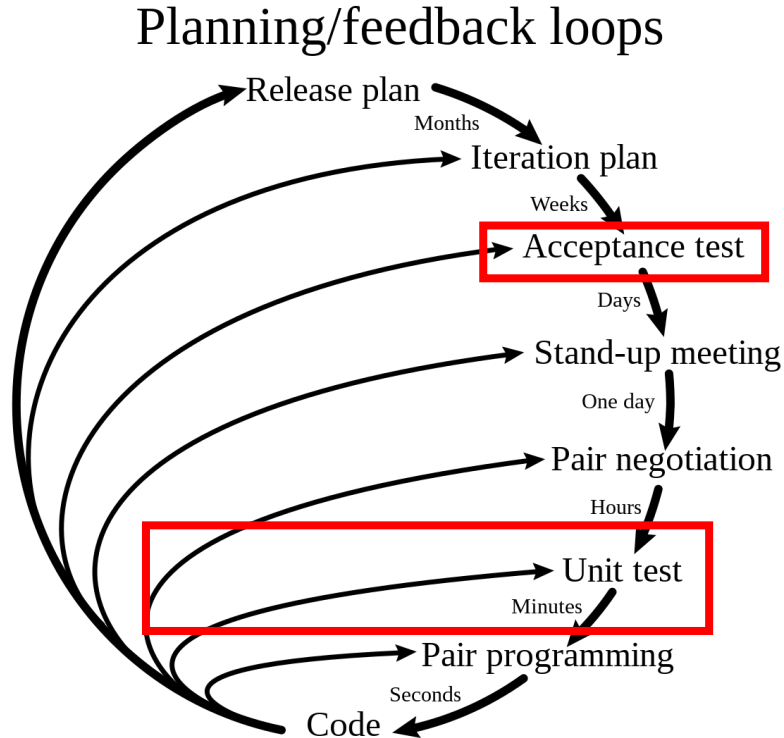


Agile



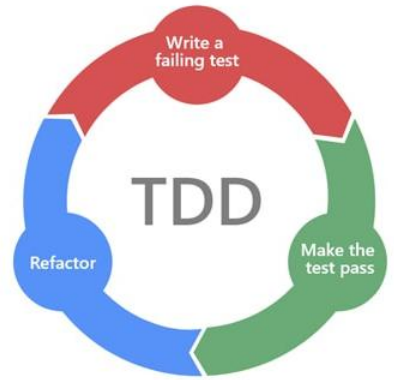
Plan-driven

Extreme Programming



Test-driven Development

1. Write test
2. Check that the newly-written test fails
3. Write the simplest code that passes the new test
4. All tests should now pass
5. Refactor as needed





Test-driven Development: Benefits

- Testable code
- High-quality tests and tested
- Avoids “feature creep”
- Increases confidence in the code

Acceptance Criteria in User Stories

Front

Users can view information about each job that is matched by a search.

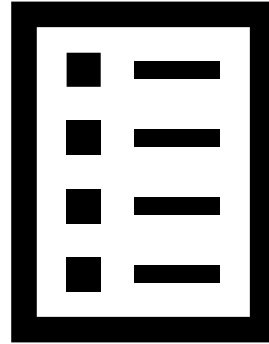
Back

Try it with an empty job description.
Try it with a really long job description.
Try it with a missing salary.
Try it with a six-digit salary.

Requirements Specification



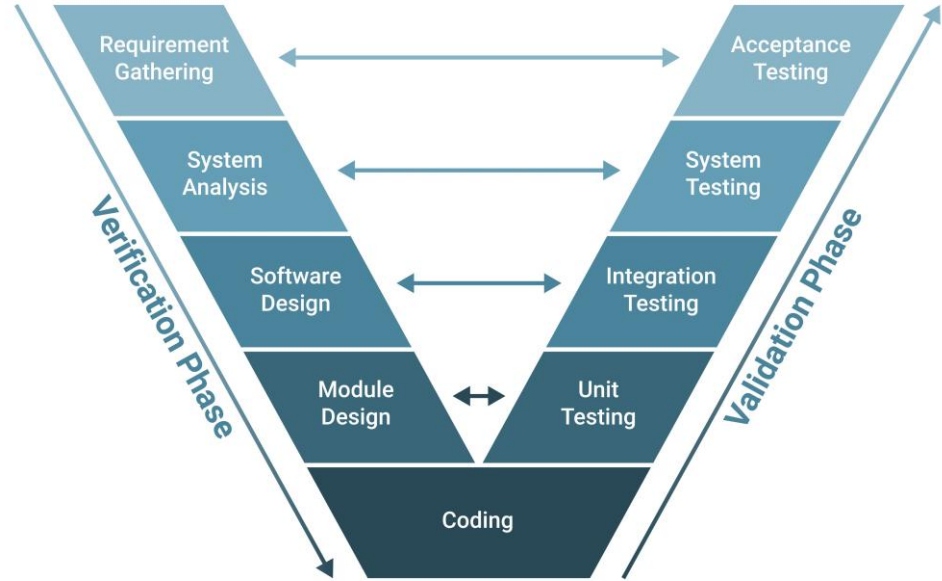
Agile



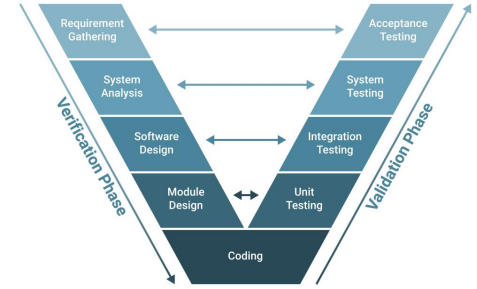
Plan-driven

V-Model

- Extension to the waterfall model
- Testing is planned together with the other phases
 - For example, during requirements engineering, the acceptance tests are derived



Verification vs. Validation



- **Verification**

- Checking that the software conforms to its specification
- "*Are we building the product **right***"

- **Validation**

- The software should do **what the user really requires**
- "*Are we building the **right product***"



(User) Acceptance Testing in Plan-Driven Approaches

- ***User Acceptance Testing (UAT)***: focuses on validation, that is, checking whether the system meets the users' requirements
 - Often involves the customer
- Explicit phase in plan-driven development, after the system has been implemented and tested



Black- and White-box Approaches

Black-box: no internal information

- For example, based on the requirements or a function's documentation
- “Specification-based testing”

White-box: internal information

- For example, based on a function's implementation
- “Structural testing”



Summary and Takeaways

- Testing for finding bugs and validating requirements
- Unit tests, integration tests, and system tests
- Agile: Extreme Programming and Acceptance Criteria
- Plan-driven: V-Model

Sources and References

